

Kernel Reconstruction in Convolutional Neural Networks

(ResNets for Optical Flow)

Supervisor:

PD Dr. Stephan Schmidt

Group M3:

Surit Nandy

Harsh Gandhi

Huy Tran

Thejas Thirthalingaiah

March 24, 2024

Contents

1	Abstract	3
2	Introduction	3
3	Literature Review	4
4	Tools and Technologies	5
5	Problems Considered	5
5.1	Known Kernels	5
5.1.1	Perform convolution with a known kernel	5
5.1.2	Perform convolution with a custom kernel	10
5.1.3	Results	12
5.2	Kernel Recovery with Machine Learning	15
5.2.1	Dataset Preparation	15
5.2.2	Model Implementation	15
5.2.3	Training	16
5.2.4	Evaluation	16
5.2.5	Results	18
6	Future Extension	19
7	Conclusion	20

1 Abstract

When we are dealing with image processing, we generally utilize a small matrix, known as kernel, convolution matrix, or mask. These are employed for a multitude of tasks, such as Gaussian blur, sharpen, and detect edges, and also, we can customize our kernel based on our need. In simpler terms, the kernel acts as a function for each pixel in the resulting image, derived from the surrounding pixels, including its own, in the original image. Convolution Neural Networks have emerged as a powerful tool in the field of computer vision and image processing. By applying a known kernel to an image, we have used Convolution Neural Networks to generate a convoluted image that highlights specific features. Also, we have generated a convoluted image from the given image and reconstructed a kernel from it. Moreover, through Convolution Neural Networks, we perceived the thought of doing further enhancements such as deconvolution by taking a convoluted image and the recovered kernel to generate the (unknown) original image. This can be used to extract valuable information from the convoluted image and to facilitate further analysis. Moreover, to obtain the transformation of the pixels, we have used the concept of a transport equation. The process of training a machine learning model on convoluted images to recover a kernel matrix from it, are discussed further.

2 Introduction

The main objective of this project is to explore and demonstrate the capabilities of Convolutional Neural Networks (CNNs) in image processing and analyzing through a series of critical operations such as, image simulation, convolution, kernel recovery, and deconvolution. By investigating these areas, this work aims to uncover the underlying principles by which CNNs interpret and manipulate image data, which ultimately contributes to advancements in image processing techniques and applications. This exploration leads to enhancing the performance of CNNs in various tasks such as image restoration, enhancement, and understanding, thereby pushing the boundaries of what is possible in the field of machine learning. The preliminary step is the image simulation which helps to generate synthetic data that can be used to train and evaluate CNN models without using extensive real-world datasets. Convolution, the core operation within CNNs, lays the groundwork to understand and manipulate image content by filtering images with kernels (or filters) to extract various features. Whereas kernel recovery on the other hand is basically the reverse engineering of the convolution process. Kernel recovery provides valuable insights, like further enhancement of model interpretability. Lastly, deconvolution, which is often used in reconstructing images, tries to reverse the effects of convolution. Deconvolution seeks to enable the restoration of images from their feature representations.

3 Literature Review

Image processing is the outcome of the modification of one pixel with respect to its neighbors. When this modification is similar in the entire image, called g , it can be mathematically defined using a second image, known as h , which defines the neighbor relationships. This results in a third image, called as f . This is what is known as convolution [1] and it is denoted with $*$:

$$f(x, y) = (g * h)(x, y) = \sum_m \sum_n g(x - m, y - n)h(m, n)$$

For the sake of simplicity, the image h is often:

- of odd size ($3 \times 3, 5 \times 5, 7 \times 7, \dots$);
- centered, i.e., the pixel with coordinates $(0, 0)$ is at the center of the image h .

The image h can be referred by different names. Depending on the context and how it is being used, we may call it like, mask, kernel, window, pattern, or point spread function (PSF). The mathematical equation for convolution involves integrating the product of two functions as one function is translated over the other. In the context of images, the kernel depicts a filter or a transformation applied to the image, and when convolution is applied with this kernel, it results into various image processing operations such as blurring, sharpening, edge detection, etc.

In image processing, convolution is performed by sliding a small array of numbers, typically a matrix of size $[3 \times 3]$ or $[5 \times 5]$, sequentially over different portions of the picture[2]. This small array of numbers is called the **kernel** as shown in Figure 1 :

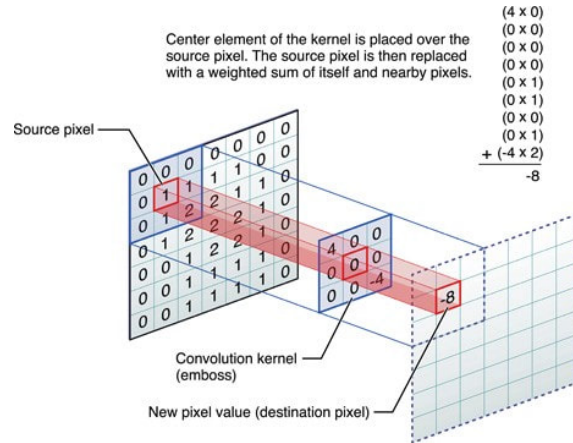


Figure 1: The Center element of the kernel is placed over the source pixel. The source pixel is then replaced with a weighted sum of itself and nearby pixels.[3]

Kernels of different sizes, containing different patterns or numbers, under the influence of convolution, produce different results. The size of a kernel is arbitrary, but mostly, the size of $[3 \times 3]$ is often used[4]. If we consider this to be an example, the convolution kernel goes over the input image and performs matrix multiplication element-wise.

The concept of kernel recovery is crucial, in Convolutional Neural Networks (CNNs) by enhancing performance and accuracy in computer vision tasks. The extracted features may become distorted, due to the loss of effectiveness of the convolutional filters within a CNN, which may impact the network's ability to recognize and classify objects accurately. Through the employment of kernel recovery techniques, it is mostly possible to restore these filters, which ultimately improves the CNN's performance and enables it to handle complex computer vision tasks more effectively.

4 Tools and Technologies

The different Software and Libraries that we have used are as follows:

- Python is a widely used programming language known for its readability and versatility. We commonly use it for web development, software development, mathematics, and system scripting. Python's simple syntax and large standard library make it suitable for various tasks, from building websites to automating processes and conducting data analysis.
- PyTorch is a popular deep learning framework built for training neural networks. We extensively use it for tasks such as image processing, natural language processing, and more. Additionally, the torchvision library, which is part of PyTorch, provides utilities for image and video processing, including datasets, transforms, and pretrained models.
- In the context of image processing, we often use Python in conjunction with libraries such as matplotlib for creating visualizations, PIL (Python Imaging Library) for opening and manipulating image files, NumPy for scientific computing, and scipy for technical computing.
- Furthermore, we utilize IPython widgets for interactive computing in the Jupyter Notebook environment.

The use of Python and its associated libraries and frameworks has become prevalent in various fields, including data science, machine learning, web development, and scientific computing. Its ease of use and extensive community support have contributed to its widespread adoption across different industries and domains.

5 Problems Considered

5.1 Known Kernels

Objective :

The aim is to explore the effect of applying specific convolution kernels on images, particularly how these kernels change image features like textures, edges, and overall sharpness. This exploration aims to illustrate the basic principles of image processing through convolution, emphasizing the mathematical relationships between kernels and their impact on image characteristics.

5.1.1 Perform convolution with a known kernel

Image Selection

We chose images without settled elements, favoring those with changing backgrounds and diverse objects. This approach guarantees that the picked images will way better show the convolution impacts. Having diverse textures and edges in the images helps to assess how well the convolution kernels work. We made sure that the images had high enough resolution to keep the details after convolution. This ensures that we can accurately measure the impact of the kernels.

Image Preprocessing

PyTorch and PIL

In our code implementation, we have begun by importing the necessary libraries, including PyTorch for tensor operations and PIL for image handling.

Image Transformation

Image transformation is applied by utilizing PyTorch's transforms. Compose with transforms.ToTensor(), converting the PIL image into a PyTorch tensor. Further, This tensor added a batch dimension using unsqueeze(0) to fit the input prerequisites of convolution operations in PyTorch.

Some Common Convolution Kernels:

Gaussian Blur: Here we are utilizing a Gaussian kernel which is defined by an equation [5],[6]:

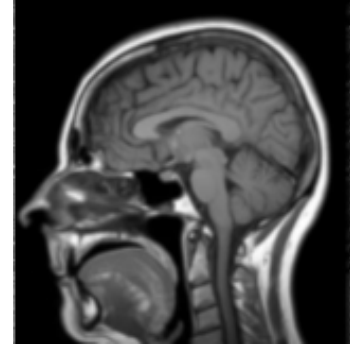
$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$, where σ is the standard deviation controlling the extent of blurring. This kernel helps in reducing image noise and detail. We have shown the feature of Gaussian Blur in the Figure 2:



(a) Original Image [3]

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

(b) Gaussian Kernel [7]



(c) Gaussian Blur Image [3]

Figure 2: Applying a Gaussian kernel to the original image results in a Gaussian blur image.

Sharpen: From the below Figure 3, we see that it employs a kernel that enhances differences in adjacent pixel values, amplifying edges and fine details to make the image appear more distinct.

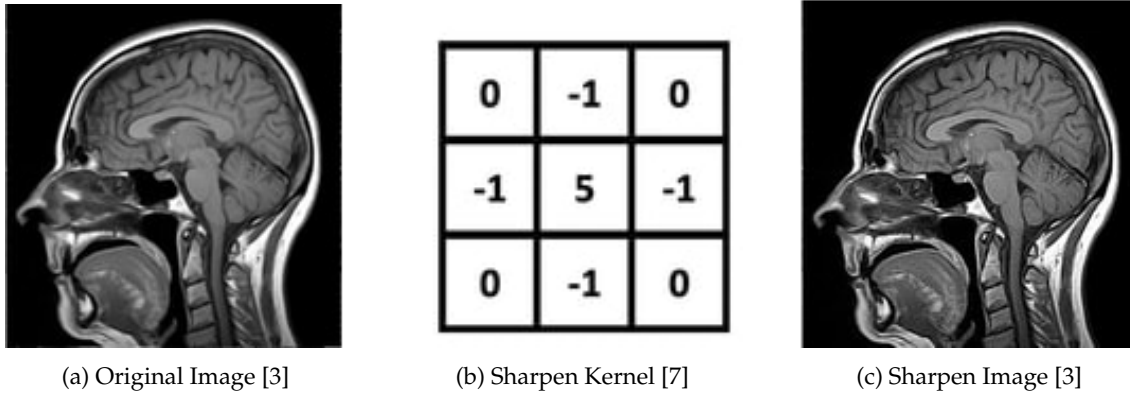


Figure 3: Applying a Sharpen kernel to the original image results in a Sharpen image.

Ridge: Designed to emphasize ridge-like structures within the image, such as edges, by enhancing contrast along these features.

Edge Detection: Here we applied kernels that highlight the spatial gradients in the image, making the edges more pronounced, as seen in the below Figure 4:

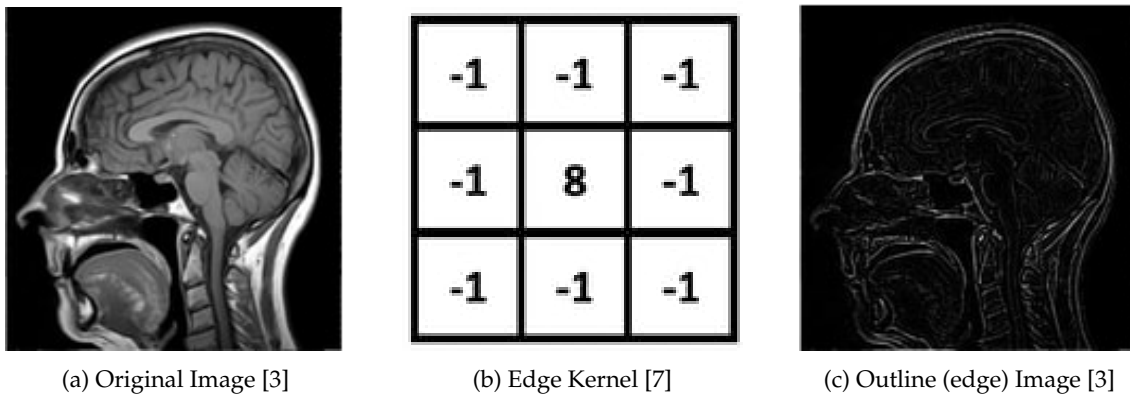


Figure 4: Applying an Edge kernel to the original image results in an Outline(Edge) image.

Identity: A basic kernel that leaves the image unchanged, serving as a control to highlight the effects of other kernels. In other words, just the starting image itself as shown in Figure 5:

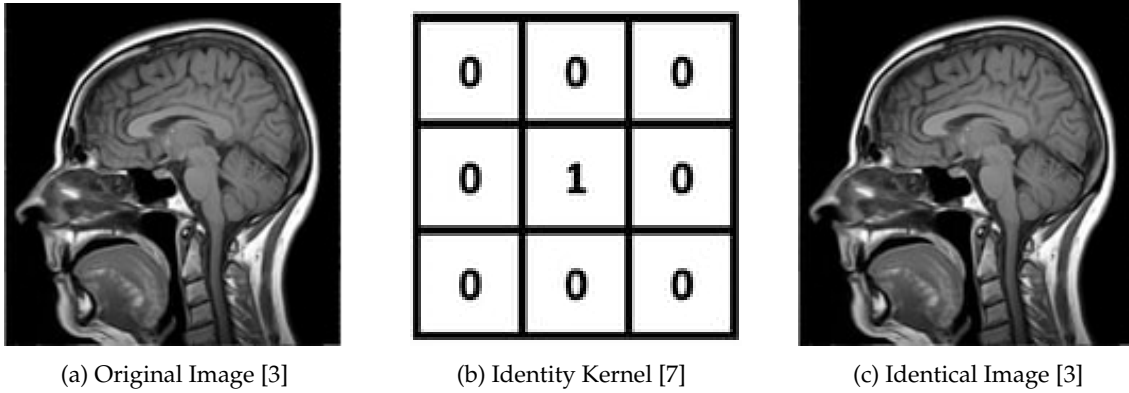


Figure 5: Applying an Identity kernel to the original image results in an Identical (to original) image.

Analysis and Evaluation

Visual Comparison: We assessed the visual differences between the original and convolved images, focusing on the changes in texture, edge definition, and overall image clarity.

Code snippet :

The below algorithm shows the functionality of generating a convoluted image from a known kernel.

Known Kernel Convolution

```
1 # Create a function to apply convolution
2 # and update the displayed images
3 class KnownConvolution(nn.Module):
4     def __init__(self, kernel, channels):
5         super(KnownConvolution, self).__init__()
6         self.channels = channels
7         self.conv_layers = nn.ModuleList([nn.Conv2d(1, 1, kernel_size
8             =kernel.shape[0], bias=False) for _ in range(self.channels
9             )])
10
11         # Set weights for each layer
12         for layer in self.conv_layers:
13             layer.weight = nn.Parameter(torch.from_numpy(kernel).
14                 unsqueeze(0).unsqueeze(0))
15             layer.weight.requires_grad = False
16
17     def forward(self, x):
18         # Perform convolution for each channel
19         convolved_channels = [layer(x[:, i:i+1, :, :]) for i, layer
20             in enumerate(self.conv_layers)]
21         return torch.cat(convolved_channels, dim=1)
22
23 # Function to apply convolution and update the displayed images
24 def apply_convolution(_):
25     global modified_image, dynamic_kernel
26
27     # Get the values from the known kernel text inputs
28     dynamic_kernel = np.array([[known_kernel_text_inputs[i][j].value
29         for j in range(3)] for i in range(3)], dtype=np.float32)
30     known_convolution = KnownConvolution(dynamic_kernel, channels=3)
31
32     # Apply convolution
33     with torch.no_grad():
34         result = known_convolution(input_tensor)
35         modified_image = result.clone()
36
37     with output_image:
38         clear_output(wait=True)
```

Algorithm : Known Kernel Convolution

Explanation:

“KnownKernelConvolution class”:

This class showcases how to apply a specific type of image transformation. While implementing this class, we specified the transformation type and the number of variations it should consider to perform convolution. When utilized, it applies the designated transformation intelligently. In a nutshell: It's all about using a predefined method to change images in a specific, controlled manner.

“apply convolution function”:

This function is like the hands-on component that executes the image transformation using our specified class. It doesn't require any special instructions, it has knowledge of what to do with the current image and the custom transformation matrix. The transformation matrix can change based on which operation the user want to perform on the given image. After applying the transformation, the image gets updated with the new appearance, ready for display or further processing.

5.1.2 Perform convolution with a custom kernel

Objective:

Examine the effects of custom-designed kernel on image processing applications.

Image Selection

In order to effectively illustrate the adaptability and significance of custom convolution kernels, the images chosen are crucial. Having diverse textures and edges in the images helps to assess how well the convolution kernels work. We made sure that the images had high enough resolution to keep the details after convolution. This ensures that we can accurately measure the impact of the kernels.

Image Preprocessing

During preprocessing, the chosen images are formatted appropriately for convolution processes. We used PyTorch, in order to convert images into tensors using a composition of transforms, primarily `transforms.ToTensor()`, which scales the pixel values to a range in $[0, 1]$. Moreover, images are resized as necessary to preserve uniform dimensions throughout the dataset.

Kernel Application

Custom Kernel Creation: The core of this case study is the use of custom kernels for convolution. Exploring a vast range of effects is possible with a custom kernel, ranging from basic blurring and sharpening to more advanced filters for particular feature enhancement or suppression.

Interactive Kernel Input: To allow the real-time modification and application of user-defined kernels, we utilized Ipython Widgets, an interactive grid input. Users may enter kernel values in real time and want to see how they affect the image due to such construction, which encourages experimentation. It additionally promotes exploring various convolution matrices in addition to helping to comprehend the direct correlation between kernel design and image procession results.

Analysis and Evaluation

Visual Analysis: The primary level of analysis is visual, where we analyzed the original and convoluted images side by side. This comparison highlights the modifications made to the image's features, textures, and overall appearance by the custom kernel. Special consideration is given to the enhancement or suppression of edges, the blurring effect on textures, and the presentation of artifacts.

Code snippet :

In the below algorithm we have tried to explore the generation of a convoluted image from a custom kernel.

Custom Kernel Convolution

```
1  # Create a function to apply convolution
2  # and update the displayed images
3  class CustomConvolution(nn.Module):
4      def __init__(self, kernel, channels):
5          super(CustomConvolution, self).__init__()
6          self.channels = channels
7          self.conv_layers = nn.ModuleList([nn.Conv2d(1, 1, kernel_size
8              =kernel.shape[0], bias=False) for _ in range(self.channels
9              )])
10
11         for layer in self.conv_layers:
12             layer.weight = nn.Parameter(torch.from_numpy(kernel).
13                 unsqueeze(0).unsqueeze(0))
14             layer.weight.requires_grad = False
15
16         def forward(self, x):
17             convolved_channels = [layer(x[:, i:i+1, :, :]) for i, layer
18                 in enumerate(self.conv_layers)]
19             return torch.cat(convolved_channels, dim=1)
20
21 # Function to apply convolution and update the displayed images
22 def apply_convolution(_):
23     global modified_image, dynamic_kernel
24
25     # Get the values from the custom kernel text inputs
26     dynamic_kernel = np.array([[custom_kernel_text_inputs[i][j].
27         value for j in range(3)] for i in range(3)], dtype=np.float32
28         )
29     custom_convolution = CustomConvolution(dynamic_kernel, channels
30         =3)
31
32     # Apply convolution
33     with torch.no_grad():
34         result = custom_convolution(input_tensor)
35         modified_image = result.clone()
36
37     with output_image:
38         clear_output(wait=True)
```

Algorithm : Custom Kernel Convolution

Explanation:

“CustomConvolution class”:

This section establishes a special tool that is capable of filtering images in a specific way. It prepares various methods for image transformation based on the provided kernel and the number of channels. It ensures that these methods remain constant and unchanged during their usage.

“apply convolution function”:

This function adjusts the image transformation based on user input. It utilized the previously specialized tool to implement these modifications. After applying the transformations, it updates the image with the new appearance and prepares it for display.

5.1.3 Results

Known Kernel:

We have already discussed about some predefined convolution kernels, and we decided to choose a certain type of kernel such as Gaussian blur to perform convolution operations on the original image. We are applying only the 3x3 kernel for demonstration purposes to give consistent performance and behavior. The Figure 6 below demonstrates the input using the Gaussian blur kernel to perform convolution operation.



Figure 6: Input for convolution with known kernel

From the below output in Figure 7, we can see that in the resulting image, the image is getting blurred by applying Gaussian blur kernel. This is the expected result when we try to apply the Gaussian blur to an image, which we have tried to replicate by our own model and utilized the given kernel value to test whether our model can give the same result.



Figure 7: Output: convoluted image from known kernel case

In the output, as shown in the Figure 8 we have given the visualization of the kernel value in the form of a heat map. The colors in the boxes represent the values from the input matrix. The brighter color corresponds with the higher kernel value. In this case, the value visualization is displaying the same with Gaussian blur kernel in 3x3 matrix.

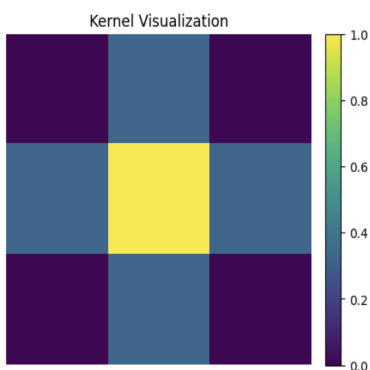


Figure 8: Output: visualization of known kernel

Custom Kernel:

For the custom kernel case, we have given the user, the ability to input their own values in the kernel within the limit of a 3x3 matrix. This allows the user to test different behaviors of how the different kernel values are applied to convolution operation which will affect the image. This allows us to experiment with more behaviors because the convolution process isn't restricted to a certain set of kernel values. For specific demonstration, as shown in Figure 9, we have used a simple kernel by putting a value of 1 in Row 2 Column 2, and keeping the all the remaining values to be 0.

Kernel Type:	Custom				
Row 1, Col 1	0	Row 1, Col 2	0	Row 1, Col 3	0
Row 2, Col 1	0	Row 2, Col 2	1	Row 2, Col 3	0
Row 3, Col 1	0	Row 3, Col 2	0	Row 3, Col 3	0

Figure 9: Input for convolution with custom kernel

By using the custom kernel, we tried to blur the image in a certain direction. As shown in the below Figure 10, we can see a certain level of blurriness from the the original image to the convoluted image, caused by the custom kernel.

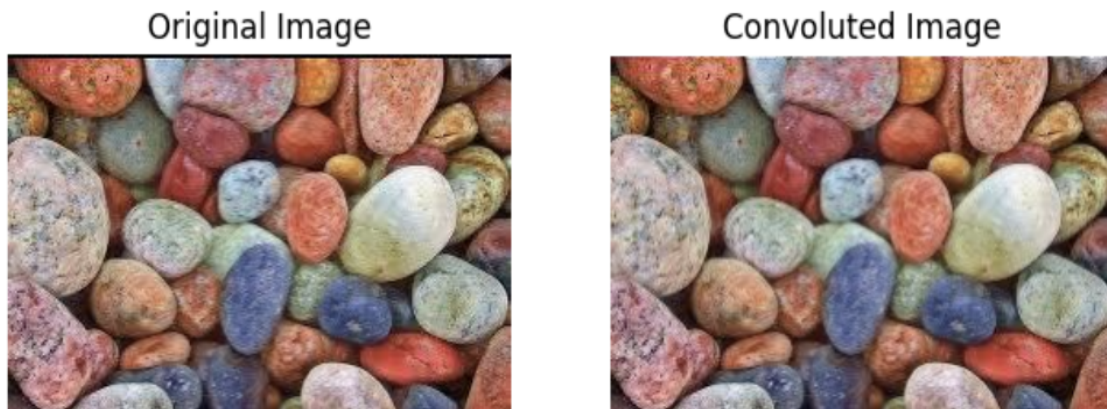


Figure 10: Output: convoluted image from custom kernel case

In the output, we have given the visualization of the kernel value in the form of a heat map. The colors in the boxes represent the values from the input matrix. The brighter color corresponds with the higher kernel value. In this case, only the middle element has the value 1 and the rest are 0, which is visualized as in the below Figure 11.

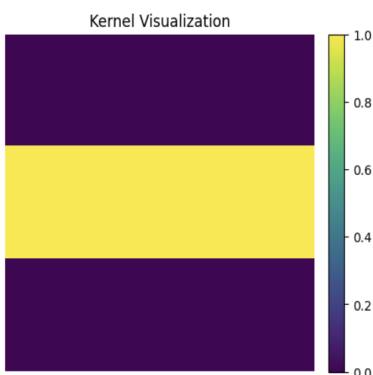


Figure 11: Output: visualization of custom kernel

5.2 Kernel Recovery with Machine Learning

Objective:

The goal is to utilize a machine learning model to determine the convolution kernel used in generating a convoluted image from its original counterpart. This entails training a convolution neural network (CNN) capable of approximating the kernel by learning from sets of original and convoluted image pairs. We explored kernel recovery by using machine learning, specifically focusing on inferring the convolution kernel from pairs of original and convoluted images. This approach is significant for understanding and reverse-engineering the impacts of convolution operations on images, which can prove crucial in diverse applications such as image restoration and analysis.

5.2.1 Dataset Preparation

Image Loading: We have utilized the Python Imaging Library (PIL) to load the original and convoluted images. This stage is crucial to ensure that the model trains on accurately paired images, which are essential for learning the convolution operation that has been applied.

Image Preprocessing: We converted both images to gray-scale to simplify the learning process, focusing on intensity patterns rather than color. This was achieved through a transformation pipeline using “transforms.Compose”, incorporating “transforms.Grayscale” and “transforms.ToTensor()”, which convert the images into tensor format suitable for processing with PyTorch.

5.2.2 Model Implementation

CNN Architecture: We defined a basic CNN architecture, KernelRecoveryCNN, which comprises a single convolution layer with a 3x3 kernel. This planned choice is a purposefulness to mimic the preparation of applying a convolution operation with a small kernel, reflecting common practices in image processing.

Forward Pass: The model’s forward method applies this convolution layer to the input tensor,

effectively learning to replicate the convolution operation that was utilized to generate the convoluted image from the original image.

5.2.3 Training

Loss Function and Optimizer: We have used the Mean Squared Error (MSE) loss [8] in the equation 5.1 to quantify the difference between the output of the CNN and the convoluted image tensor. A Stochastic Gradient Descent (SGD) optimizer, is shown in the equation 5.2. But in our case it actually acts as a standard gradient descent, which has been employed to adjust the model parameters, aiming to minimize this loss over training iterations. The Mean Squared Error (MSE) is calculated using the formula:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - p_i)^2 \quad (5.1)$$

where:

y_i : is the i th observed value,

p_i : is the corresponding predicted value for y_i ,

n : is the number of observations.

$\sum$: indicates that a summation is performed over all values of i .

The gradient descent is given by [9]:

$$\theta_{j+1} = \theta_j - \alpha \nabla K(\theta_j) \quad (5.2)$$

where:

θ_j : Parameters at iteration j

θ_{j+1} : Updated parameters at iteration $j + 1$

α : Learning rate (step size)

$\nabla K(\theta_j)$: Gradient of the cost function K with respect to θ at iteration j

Training Loop: Executed a training loop for a predefined number of epochs, where we iterated each epoch involving a forward pass (applying the learned convolution to the original image), loss calculation, back-propagation, and an optimization step. This process iteratively refines the model's weights, specifically the kernel values, to reduce the loss.

5.2.4 Evaluation

Performance Measurement: After iterating through the training loop, we evaluated the model by comparing its output (the reconstructed image) against the convoluted image tensor using the MSE loss function. This evaluation step provides a quantitative measure of how well the model has learned to replicate the convolution process.

Code snippet :

The below algorithm helps us to create a kernel recovery model from the original and convoluted images.

Kernel Recovery Model

```
1  # Convert the original and convoluted images to tensors
2  transform = transforms.Compose([
3      transforms.Grayscale(num_output_channels=1),
4      transforms.ToTensor()
5  ])
6
7  original_image_tensor = transform(original_img)
8  convoluted_image_tensor = transform(convoluted_img)
9
10 class KernelRecoveryCNN(nn.Module):
11     def __init__(self):
12         super(KernelRecoveryCNN, self).__init__()
13         self.conv1 = nn.Conv2d(1, 1, kernel_size=3, bias=False)
14
15     def forward(self, x):
16         x = self.conv1(x)
17         return x
18
19 # Initialize the Kernel Recovery CNN model
20 model = KernelRecoveryCNN()
```

Algorithm : Kernel Recovery Model

Explanation:

Before we apply the kernel recovery model, we perform pre-processing of the input images. The input images are the original image and the convoluted image that was affected by the convolution operation. In the pre-processing step, we use gray scale with a number of output channels as 1 to convert the input images into tensors. This helps the model understand the input and perform the kernel recovery on the image tensor.

In our model, with the implementation of PyTorch, we perform the kernel recovery with 1 layer of convolution. In PyTorch, the “nn.Conv2d” is a function in torch.nn module, which we have used to create a 2-dimensional convolution layer. In our model, “1,1” indicates the input, whereas the output channel is only 1. We have kept the kernel size to be 3, as we perform the operation specifically in a 3x3 kernel matrix. We specified the bias to be false as we have no bias in this case.

Inside the forward function, we perform a forward pass computation in CNN. Every time the model receives the input “x” as shown in the above algorithm, it will apply the configured convolution layer to the input. It gives the output containing the transform representation of the input tensor.

5.2.5 Results

Figure 12 tells us about the evaluation metric, measured in the the form of Mean Squared Error.

Mean Squared Error (MSE): 0.00010579529771348462

Figure 12: Quantified difference between the output of the CNN and the convoluted image tensor

For the current case, the input will be a pair of original and convoluted images as shown in the below Figure 13. With the given input we expect the model to learn from both the images and reconstruct the kernel that caused this convolution operation.

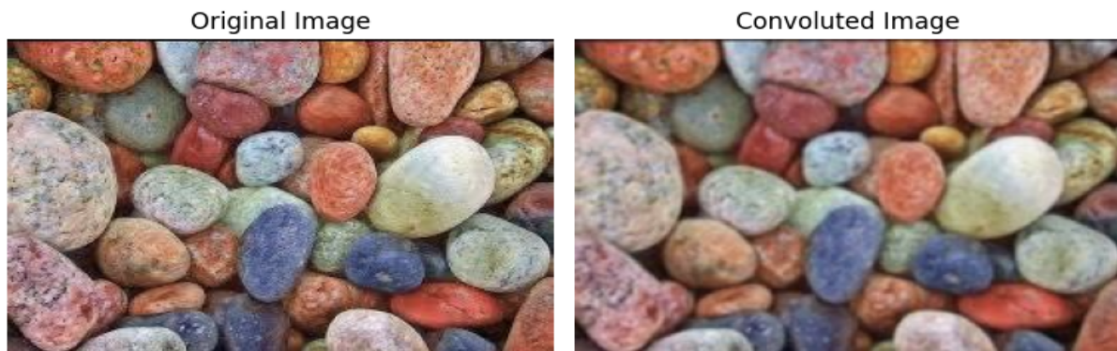


Figure 13: Input of kernel recovery

In the Figure 14, we have given the visualization of the kernel value in the form of a heat map. The colors in the boxes matches with the values from the recovered kernel matrix. The brighter color corresponds with the higher kernel value. In this case, we have recovered the kernel with have same rule distribution like value at the centre is greater than value at the corners, which is ultimately greater than the rest of the values, i.e. Value at the center > Value at 4 corners > Rest Values.

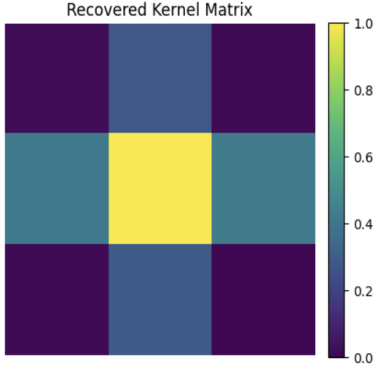


Figure 14: Output: Recovered kernel heat map

In the training phase after iterating with 10000 epochs we were able to get the recovered kernel. The values are different from the original input but it still gives the same result when we use this recovered kernel to perform convolution operation. Figure 15 gives us the recovered kernel matrix.

Recovered 3x3 Kernel Matrix:

Row 1, Col 1	0.0746078	Row 1, Col 2	0.1116813	Row 1, Col 3	0.0739759
Row 2, Col 1	0.1310675	Row 2, Col 2	0.2139744	Row 2, Col 3	0.1326063
Row 3, Col 1	0.0737771	Row 3, Col 2	0.1131286	Row 3, Col 3	0.0719834

Figure 15: Output: Recovered kernel

6 Future Extension

Our main goals in the extension phase are to improve the current kernel recovery model and accelerate the deconvolution method's completion process.

First, we address improving kernel recovery precision, which is a crucial component in the development of the kernel recovery model. To increase the value of the kernel, this means adding more layers to the model. Furthermore, our goal is to make the model more flexible to other kernel sizes, like 5x5, so that it remains effective in a variety of dynamic settings. Moreover, it is essential to optimize the loss function because it might call for the use of better parameter fitting methods and loss functions, like ADAM optimization.

Performing image deconvolution would have been our goal for our second task in the future. In order to do this, you will need to submit both the convoluted kernel and the convoluted picture. The goal is to extract an image that, when applied to the same convoluted kernel, produces the same convolutional results as the input. We would have used an iterative strategy to create a mathematical formula, which would have allowed the model to use the formula to reconstruct an image as closely as feasible to the unknown image "X".

7 Conclusion

This research aims to use the PyTorch framework in conjunction with convolutional neural networks (CNNs) for kernel recovery and picture editing. Our research endeavors to explore the inner workings of CNNs in order to create machine learning models that can handle challenging tasks such as deconvolution and kernel recovery. Although the results of our machine learning model for kernel recovery are encouraging, there is still space for improvement. Furthermore, we are excited to tackle the difficulties involved in picture deconvolution since it offers a chance to fully investigate the capabilities of CNNs.

References

- [1] Jähne, B., (2005). Book Name: *Digital Image Processing*. Springer.
- [2] Ganesh, P., (2019:1-12). Medium Story Name: *Types of convolution kernels: simplified*. Toward Data Science
- [3] Accessed on 15-03-2024. <https://mriquestions.com/what-is-convolution.html>
- [4] Ludwig, J. , Course Name: *GEOG 481/581 : Satellite Digital Image Analysis*. Image Convolution. Portland State University
- [5] Accessed on 15-03-2024. <https://pages.stat.wisc.edu/~mchung/teaching/MIA/reading/diffusion.gaussian.kernel.pdf.pdf>
- [6] https://en.wikipedia.org/wiki/Gaussian_filter
- [7] Accessed on 15-03-2024. [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))
- [8] Stewart K. , Encyclopedia Britannica: *Mean Squared Error, Mathematics*. <https://www.britannica.com/science/mean-squared-error>
- [9] Accessed on 15-03-2024. https://en.wikipedia.org/wiki/Gradient_descent